



NumPy Handbook

numpy #python #datascience

Why Use NumPy?

Python lists are flexible but **slow** for numerical computing because they:

- Store elements as **pointers** instead of a contiguous block of memory
- Lack **vectorized operations** — **rely on loops** instead
- Have **significant overhead** due to **dynamic typing**

NumPy's Superpowers

Feature	Benefit
C-optimized backend	Faster than Python lists
Efficient storage	Uses less memory
Vectorized operations	No explicit loops needed
Built-in math functions	Ready to use out of the box

Performance Comparison

```
import numpy as np
import time

size = 1_000_000
list1 = list(range(size))
list2 = list(range(size))

# Python list – slow
start = time.time()
result = [x + y for x, y in zip(list1, list2)]
print("Python list addition time:", time.time() - start)

# NumPy array – fast
arr1 = np.array(list1)
arr2 = np.array(list2)
start = time.time()
result = arr1 + arr2 # Vectorized operation
print("NumPy array addition time:", time.time() - start)
```

[!tip] Key Takeaway

NumPy is significantly faster because it performs operations in **C**, avoiding Python's interpreter overhead.

Memory Efficiency

```
import sys

list_data = list(range(1000))
numpy_data = np.array(list_data)

print("Python list size:", sys.getsizeof(list_data) * len(list_data), "bytes")
print("NumPy array size:", numpy_data.nbytes, "bytes")
```

Creating NumPy Arrays

From Python Lists

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])      # 1D array
arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array

print(arr1)      # [1 2 3 4 5]
print(arr2)      # [[1 2 3]
                  #  [4 5 6]]
```

[!warning] Unlike lists, all elements must share the **same data type**.

Creating Arrays from Scratch

```
np.zeros((3, 3))      # 3x3 array of zeros
np.ones((2, 4))       # 2x4 array of ones
np.full((2, 2), 7)    # 2x2 array filled with 7
np.eye(4)             # 4x4 identity matrix
np.arange(1, 10, 2)   # [1, 3, 5, 7, 9] – like range()
np.linspace(0, 1, 5)  # [0. 0.25 0.5 0.75 1.] – evenly spaced
```

Checking Array Properties

```
arr = np.array([[10, 20, 30], [40, 50, 60]])

print("Shape:", arr.shape)      # (2, 3) → 2 rows, 3 columns
print("Size:", arr.size)        # 6 → total elements
print("Dimensions:", arr.ndim)  # 2 → 2D array
print("Data type:", arr.dtype)  # int64
```

Changing Data Types

```
arr = np.array([1, 2, 3], dtype=np.float32) # Explicit type
arr_int = arr.astype(np.int32)             # Convert float → int
```

Reshaping and Flattening

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

reshaped = arr.reshape((3, 2)) # Change shape → (3, 2)
flattened = arr.flatten()      # Convert 2D → 1D: [1 2 3 4 5 6]
```

Indexing & Slicing

Basic Indexing

```
arr = np.array([10, 20, 30, 40])

print(arr[0]) # 10 – first element
print(arr[-1]) # 40 – last element
```

Slicing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4]) # [20 30 40] – index 1 to 3
print(arr[:3]) # [10 20 30] – first 3 elements
print(arr[::2]) # [10 30 50] – every 2nd element
```

[!warning] Slicing returns a **view**, not a copy!

Changes to the slice affect the **original** array.

```
python sliced = arr[1:4] sliced[0] = 999 print(arr) # [10 999 30 40 50] ← original changed!
```

Use `.copy()` if you need an independent copy.

Fancy Indexing

```
arr = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
print(arr[idx]) # [10 30 50]
```

Boolean Masking

```
arr = np.array([10, 20, 30, 40, 50])
mask = arr > 25
print(arr[mask]) # [30 40 50]
```

[!tip] Boolean masking is a powerful way to **filter large datasets efficiently!**

Multidimensional Indexing & Axes

Understanding Axes

Array	Axes
1D	axis 0 = elements
2D	axis 0 = rows, axis 1 = columns
3D	axis 0 = depth, axis 1 = rows, axis 2 = columns

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

print(np.sum(arr, axis=0)) # [12 15 18] – column-wise sum
print(np.sum(arr, axis=1)) # [ 6 15 24] – row-wise sum
```

2D Indexing

```
print(arr[1, 2])          # Row 1, Column 2 → 6
print(arr[0:2, 1:3])      # First 2 rows, last 2 columns
                          # [[2 3]
                          #  [5 6]]
```

3D Indexing

```
arr3D = np.array([[[1, 2, 3], [4, 5, 6]],
                  [[7, 8, 9], [10, 11, 12]]])

print(arr3D.shape)        # (2, 2, 3) → (depth, rows, columns)
print(arr3D[0, 1, 2])     # 6 – sheet 0, row 1, col 2
print(arr3D[:, 0, :])     # First row from all sheets
```

Selecting & Modifying Along an Axis

```
first_col = arr[:, 0]     # All rows, column 0 → [1 4 7]

arr[:, 1] = 0             # Replace entire column 1 with 0
# [[1 0 3]
#  [4 0 6]
#  [7 0 9]]
```

Data Types (dtypes)

NumPy arrays are **homogeneous** — all elements must be the same type.

Common dtypes

dtype	Description
<code>int32</code> , <code>int64</code>	Integers (different bit sizes)
<code>float32</code> , <code>float64</code>	Floating-point (different precision)
<code>bool</code>	Boolean
<code>complex64</code> , <code>complex128</code>	Complex numbers
<code>object</code>	Python objects (avoid if possible)
<code>U</code> (Unicode)	Strings

Type Conversion with `.astype()`

```
arr = np.array([1.5, 2.7, 3.9])
arr_int = arr.astype(np.int32) # [1 2 3]
```

Memory Impact

```
arr_int64 = np.array([1, 2, 3], dtype=np.int64)
arr_int32 = np.array([1, 2, 3], dtype=np.int32)

print(arr_int64.nbytes) # 24 bytes (3 × 8 bytes)
print(arr_int32.nbytes) # 12 bytes (3 × 4 bytes)
```

Special Types

```
# Strings
arr = np.array(['apple', 'banana', 'cherry'], dtype='U10')

# Complex numbers
arr = np.array([1+2j, 3+4j, 5+6j], dtype='complex128')

# Objects (use sparingly – slow!)
arr = np.array(['a': 1], [1, 2, 3], 'hello'], dtype=object)
```

[!note] Choosing the Right dtype

- **Optimize memory** → use the smallest type that fits your data
- **Maximize performance** → smaller types are processed faster
- **Preserve precision** → don't downcast if decimal accuracy matters

Broadcasting & Vectorization

Why Loops Are Slow

Python loops are slow because:

- The interpreter re-evaluates logic every iteration
- High overhead: function calls, memory access, index management

Vectorization (The Fix)

Instead of looping, apply the operation to the **entire array at once**.

```
arr = np.array([1, 2, 3, 4, 5])

# Slow – loop
result = [x**2 for x in arr]

# Fast – vectorized
result = arr ** 2 # [1 4 9 16 25]
```


Why faster?

- Implemented in **C** (compiled)
- Uses **SIMD** (Single Instruction, Multiple Data) — parallel processing

Broadcasting with a Scalar

```
arr = np.array([1, 2, 3, 4, 5])
result = arr + 10    # [11 12 13 14 15]
# The scalar 10 is "broadcast" across all elements — no extra memory used
```

Broadcasting with Arrays of Different Shapes

```
arr1 = np.array([[1, 2, 3],
                  [4, 5, 6]])
arr2 = np.array([1, 2, 3])

result = arr1 + arr2 # arr2 broadcast across each row
# [[2 4 6]
#   [5 7 9]]
```

Broadcasting Rules

1. Trailing dimensions must match, or one of them must be 1
2. NumPy stretches the smaller array — **no data is copied**

Real-World Example: Normalizing Data

```
data = np.array([[10, 20, 30],
                 [15, 25, 35],
                 [20, 30, 40],
                 [25, 35, 45],
                 [30, 40, 50]])

mean = data.mean(axis=0) # Column-wise mean
std = data.std(axis=0) # Column-wise std deviation

normalized_data = (data - mean) / std # Broadcasting handles it all
print(normalized_data)
```

Built-in Mathematical Functions

Statistics

Function	Description
<code>np.mean(arr)</code>	Mean (average)
<code>np.median(arr)</code>	Median
<code>np.std(arr)</code>	Standard deviation
<code>np.var(arr)</code>	Variance
<code>np.min(arr)</code>	Minimum value
<code>np.max(arr)</code>	Maximum value
<code>np.percentile(arr, 50)</code>	Percentile (e.g. 50th = median)

Aggregation

Function	Description
<code>np.sum(arr)</code>	Sum of all elements
<code>np.prod(arr)</code>	Product of all elements
<code>np.cumsum(arr)</code>	Cumulative sum
<code>np.diff(arr)</code>	Differences between elements

Indexing Utilities

Function	Description
<code>np.argmin(arr)</code>	Index of minimum value
<code>np.argmax(arr)</code>	Index of maximum value
<code>np.unique(arr)</code>	Unique elements
<code>np.corrcoef(arr1, arr2)</code>	Correlation coefficient matrix

Math & Calculus

Function	Description
<code>np.log(arr)</code>	Natural logarithm
<code>np.exp(arr)</code>	Exponential (e^x)
<code>np.linspace(0, 10, 5)</code>	5 evenly spaced values from 0 to 10

Practice Exercises

Chapter 1 — Why NumPy?

- [] Create a NumPy array with values from 10 to 100 and print its shape
- [] Compare time taken to multiply two Python lists vs. two NumPy arrays

- [] Find the memory size of a NumPy array with 1 million elements

Chapter 2 — Creating Arrays

- [] Create a 3×3 array filled with random numbers and print its shape
- [] Convert an array of floats `[1.1, 2.2, 3.3]` into integers
- [] Use fancy indexing to extract even numbers from `[1, 2, 3, 4, 5, 6]`
- [] Reshape a 1D array of size 9 into a 3×3 matrix
- [] Use boolean masking to filter numbers greater than 50 in an array